Paradigmas de Programación [1]

Fundamentos de programación funcional [2]

2do cuatrimestre de 2025 [3]

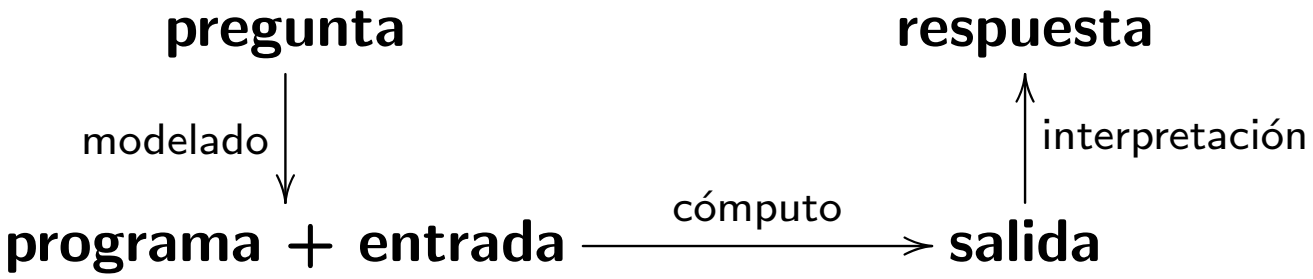
Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Fundamentos de programación funcional 1

Ejercicios 2

Programación funcional 1

Un problema central en computación es el de procesar información: 2



La programación funcional consiste en definir funciones y aplicarlas para procesar información. 3

Las "funciones" son verdaderamente funciones (parciales): 4

- Aplicar una función no tiene efectos secundarios. 5
- A una misma entrada corresponde siempre la misma salida.
- Las estructuras de datos son inmutables.

Las funciones son datos como cualquier otro: 6

- Se pueden pasar como parámetros. 7
- Se pueden devolver como resultados.
- Pueden formar parte de estructuras de datos.
  (Ej. árbol binario en cuyos nodos hay funciones).

Programación funcional 1

Un programa funcional está dado por un conjunto de ecuaciones: 2

Ejemplo 3

```
longitud []       = 0
longitud (x : xs) = 1 + longitud xs
```
4

```
   longitud [10, 20, 30]
≡  longitud (10 : (20 : (30 : [])))
=  1 + longitud (20 : (30 : []))
=  1 + (1 + (longitud (30 : [])))
=  1 + (1 + (1 + longitud []))
=  1 + (1 + (1 + 0))
=  1 + (1 + 1)
=  1 + 2
=  3
```
5

Expresiones 1

Las **expresiones** son secuencias de símbolos que sirven para representar datos, funciones y funciones aplicadas a los datos. (Recordemos: las funciones también son datos). 2

Una expresión puede ser: 3

1. Un **constructor**: 4

   `True  False  []  (:)  0  1  2  ...` 5

2. Una **variable**: 6

   `longitud  ordenar  x  xs  (+)  (*)  ...`

3. La **aplicación** de una expresión a otra: 7

   ```
   ordenar lista
   not True
   not (not True)
   (+) 1
   ((+) 1) (alCuadrado 5)
   ```
   8

4. También hay expresiones de otras formas, como veremos. Las tres de arriba son las fundamentales. 9

Expresiones 1

Convenimos en que la aplicación es **asociativa a izquierda**: 2

```
f x y       ≡  (f x) y          ≢  f (x y)
f a b c d   ≡  (((f a) b) c) d
```

Ejemplo 3

```
    [1, 2]                      4
≡   1 : [2]
≡   (:) 1 [2]
≡   ((:) 1) [2]
≡   ((:) 1) (2 : [])
≡   ((:) 1) ((:) 2 [])
≡   ((:) 1) (((:) 2) [])
```

Expresiones

Ejemplo 1

```
sumarUno = (+) 1
```
2

```
    sumarUno (sumarUno 5)
=   ((+) 1) (sumarUno 5)
≡   1 + sumarUno 5
=   1 + ((+) 1) 5
≡   1 + (1 + 5)
=   1 + 6
=   7
```
3

Tipos [1]

Hay secuencias de símbolos que no son expresiones bien formadas. [2]

Ejemplo [3]

```
1,,2      )f x(
```

Hay expresiones que están bien formadas pero no tienen sentido. [4]

Ejemplo [5]

```
True + 1
0 1
[[], (+)]
```

[6]

Tipos 1

Un **tipo** es una especificación del invariante de un dato o de una función. 3

Ejemplo 4

```
99          :: Int
not         :: Bool -> Bool
not True    :: Bool
(+)         :: Int -> (Int -> Int)
(+) 1       :: Int -> Int
((+) 1) 2   :: Int
```

5

El tipo de una función expresa un **contrato**. 6

Tipos 1

Condiciones de tipado 2

Para que un programa esté **bien tipado**: 3

1. Todas las expresiones deben tener tipo. 4
2. Cada variable se debe usar siempre con un mismo tipo.
3. Los dos lados de una ecuación deben tener el mismo tipo.
4. El argumento de una función debe tener el tipo del dominio.
5. El resultado de una función debe tener el tipo del codominio.

$$\frac{\texttt{f :: a -> b} \qquad \texttt{x :: a}}{\texttt{f x :: b}}$$

Sólo tienen sentido los programas bien tipados. 5

No es necesario escribir explícitamente los tipos. (Inferencia). 6

Tipos 1

Convenimos en que "->" es **asociativo a derecha**: 2

```
a -> b -> c        ≡  a -> (b -> c)    ≢  (a -> b) -> c 3
a -> b -> c -> d   ≡  a -> (b -> (c -> d))
```

Ejemplo 4

```
suma4 :: Int -> Int -> Int -> Int -> Int 5
suma4 a b c d = a + b + c + d
```

Se puede pensar así: 6

```
suma4 :: Int -> (Int -> (Int -> (Int -> Int))) 7
(((suma4 a) b) c) d = a + b + c + d
```

Polimorfismo 1

Hay expresiones que tienen más de un tipo. 2
Usamos *variables de tipo* a, b, c para denotar tipos desconocidos:

```
id   ::  a -> a
[]   ::  [a]
(:)  ::  a -> [a] -> [a]
fst  ::  (a, b) -> a
snd  ::  (a, b) -> b
```

Ejemplo 4

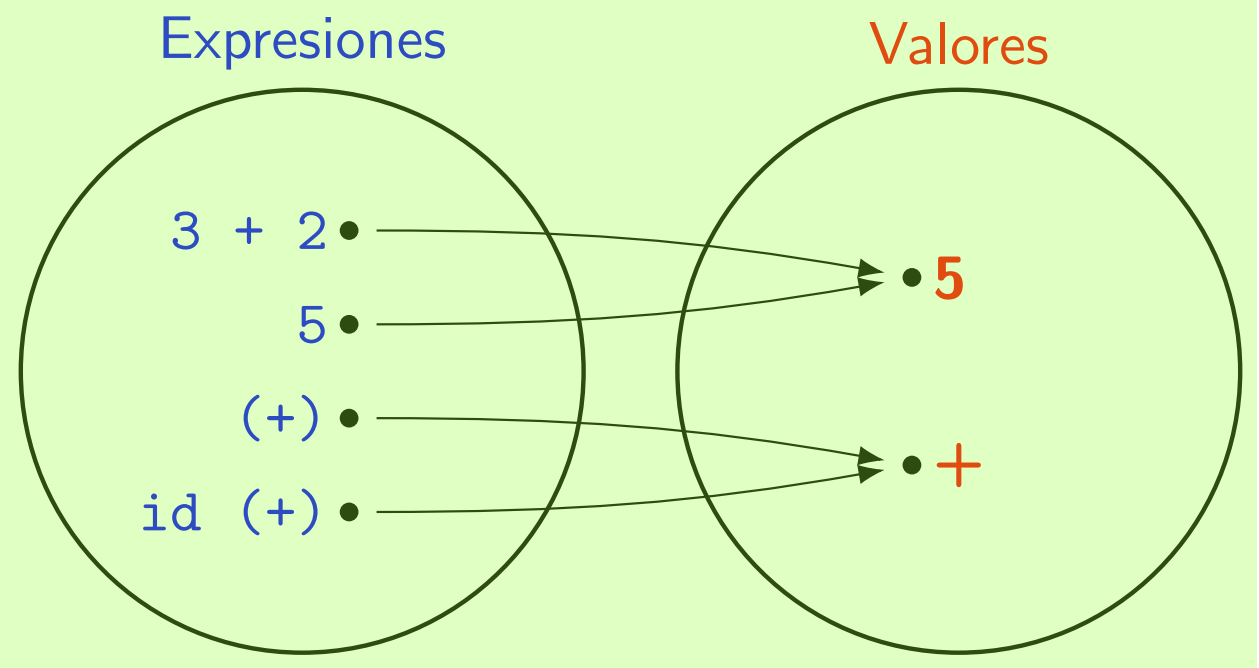
```
flip f x y = f y x
```
5

¿Qué tipo tiene `flip`?

```
   flip (:) [2, 3] 1
=  (:) 1 [2, 3]
≡  1 : [2, 3]
=  [1, 2, 3]
```
6

Modelo de cómputo 1

Dada una expresión, se computa su *valor* usando las ecuaciones: 2



3

Hay expresiones bien tipadas que no tienen valor. Ej.: `1 / 0`. 4
Decimos que dichas expresiones se indefinen o que tienen valor $\perp$.

Modelo de cómputo [1]

Un programa funcional está dado por un conjunto de ecuaciones. [2]
Más precisamente, por un conjunto de **ecuaciones orientadas**.

Una ecuación `e1 = e2` se interpreta desde dos puntos de vista: [3]

1. **Punto de vista denotacional.** [4]
   Declara que `e1` y `e2` tienen el mismo significado.
2. **Punto de vista operacional.** [5]
   Computar el valor de `e1` se reduce a computar el valor de `e2`.

Modelo de cómputo 1

El lado *izquierdo* de una ecuación no es una expresión arbitraria. 2
Debe ser una función aplicada a **patrones**.

Un patrón puede ser: 3

1. Una variable. 4
2. Un comodín _.
3. Un constructor aplicado a patrones.

El lado izquierdo no debe contener variables repetidas.

Ejemplo 5

¿Cuáles ecuaciones están sintácticamente bien formadas? 6

```
sumaPrimeros (x : y : z : _) = x + y + z
predecesor (n + 1) = n
iguales x x = True
```

7

Modelo de cómputo 1

Evaluar una expresión consiste en: 2

1. Buscar la subexpresión más externa que coincida con el lado izquierdo de una ecuación. 3
2. Reemplazar la subexpresión que coincide con el lado izquierdo de la ecuación por la expresión correspondiente al lado derecho.
3. Continuar evaluando la expresión resultante.

La evaluación se detiene cuando se da uno de los siguientes casos: 4

1. La expresión es un constructor o un constructor aplicado. 5

   `True` `(:) 1` `[1, 2, 3]` 6

2. La expresión es una función *parcialmente* aplicada. 7

   `(+)` `(+) 5` 8

3. Se alcanza un *estado de error*. 9
   Un estado de error es una expresión que no coincide con las ecuaciones que definen a la función aplicada.

Modelo de cómputo 1

Ejemplo: resultado — constructor 2

```
tail :: [a] -> [a] 3
tail (_ : xs) = xs
```

```
tail (tail [1, 2, 3]) ⇝ tail [2, 3] ⇝ [3] 4
```

Ejemplo: resultado — función parcialmente aplicada 5

```
const :: a -> b -> a 6
const x y = x
```

```
const (const 1) 2 ⇝ const 1 7
```

Modelo de cómputo 1

Ejemplo: indefinición — error 2

```
head :: [a] -> a
head (x : _) = x
```
3

```
head (head [[], [1], [1, 1]])  ⇝  head []
                               ⇝  ⊥
```
4

Ejemplo: indefinición — no terminación 5

```
loop :: Int -> a
loop n = loop (n + 1)
```
6

```
loop 0  ⇝  loop (1 + 0)
        ⇝  loop (1 + (1 + 0))
        ⇝  loop (1 + (1 + (1 + 0)))
           ...
```
7

Modelo de cómputo [1]

Ejemplo: evaluación no estricta [2]

```
indefinido :: Int
indefinido = indefinido
```
[3]

```
    head (tail [indefinido, 1, indefinido])
⇝   head [1, indefinido]
⇝   1
```
[4]

Modelo de cómputo 1

Ejemplo: listas infinitas 2

```
desde :: Int -> [Int]
desde n = n : desde (n + 1)
```
3

```
    desde 0
⇝   0 : desde 1
⇝   0 : (1 : desde 2)
⇝   0 : (1 : (2 : desde 3))  ⇝  ...
```
4

```
    head (tail (desde 0))
⇝   head (tail (0 : desde 1))
⇝   head (desde 1)
⇝   head (1 : desde 2)
⇝   1
```
5

Modelo de cómputo 1

Nota. En Haskell, el orden de las ecuaciones es relevante. 2
Si hay varias ecuaciones que coinciden se usa siempre la primera.

Ejemplo 3

```
esCorta (_ : _ : _) = False 4
esCorta _           = True
```

```
esCorta []      ⇝  True  5
esCorta [1]     ⇝  True
esCorta [1, 2]  ⇝  False
```

Funciones de orden superior 1

Definamos la composición de funciones ("`g . f`"). 2

```
(.) :: (b -> c) -> (a -> b) -> a -> c
(g . f) x = g (f x)
```

Otra forma de definirla (usando la notación "lambda"): 3

```
(.) :: (b -> c) -> (a -> b) -> a -> c
g . f = \ x -> g (f x)
```

Funciones de orden superior 1

¿Qué tienen en común las siguientes funciones? 2

```
doble L :: [Float] -> [Float]
doble L []       = []
doble L (x : xs) = x * 2 : doble L xs
```
3

```
esParL :: [Int] -> [Bool]
esParL []       = []
esParL (x : xs) = x `mod` 2 == 0 : esParL xs
```
4

```
longitudL :: [[a]] -> [Int]
longitudL []       = []
longitudL (x : xs) = length x : longitudL xs
```
5

Todas ellas siguen el esquema: 6

```
g []       = []
g (x : xs) = f x : g xs
```
7

Funciones de orden superior 1

¿Cómo se puede abstraer el esquema? 2

```
map :: (a -> b) -> [a] -> [b]
map f []       = []
map f (x : xs) = f x : map f xs
```
3

```
dobleL    xs = map (\ x -> x * 2) xs
esParL    xs = map (\ x -> x `mod` 2 == 0) xs
longitudL xs = map length xs
```
4

Otra manera: 5

```
dobleL    = map (* 2)
esParL    = map ((== 0) . (`mod` 2))
longitudL = map length
```
6

Funciones de orden superior 1

¿Qué relación hay entre las siguientes funciones? 2

```
negativos :: [Int] -> [Int]
negativos []       = []
negativos (x : xs) = if x < 0
                       then x : negativos xs
                       else negativos xs
```

3 4 5

```
noVacias :: [[a]] -> [[a]]
noVacias []       = []
noVacias (x : xs) = if not (null x)
                      then x : noVacias xs
                      else noVacias xs
```

6

Ambas siguen el esquema: 7

```
g []       = []
g (x : xs) = if p x
               then x : g xs
               else g xs
```

8

Funciones de orden superior 1

¿Cómo se puede abstraer el esquema? 2

```
filter :: (a -> Bool) -> [a] -> [a]
filter p []       = []
filter p (x : xs) = if p x
                      then x : filter p xs
                      else filter p xs
```

3

```
negativos = filter (< 0)
noVacias  = filter (not . null)
```

4

Ejercicio (tarea) 1

```
merge     :: (a -> a -> Bool) -> [a] -> [a] -> [a]
mergesort :: (a -> a -> Bool) -> [a] -> [a]
```
2

El primer parámetro es una función que determina una relación de orden total entre los elementos de tipo a. 3

Fundamentos de programación funcional [1]

Ejercicios [2]

Algunos esquemas de recursión 1

a) Definir una función: 2

```
operatoria :: (a -> a -> a) -> [a] -> a
```
3

que dada una operación binaria (que asumimos asociativa) y una lista no vacía devuelve el resultado de hacer la operación entre todos los elementos.

Informalmente: 4

$$\texttt{operatoria}\ (\bigstar)\ [x_1, x_2, \ldots, x_n] = x_1 \bigstar x_2 \bigstar \ldots \bigstar x_n.$$

b) Definir la sumatoria y la productoria como casos particulares. 5

Algunos esquemas de recursión 1

a) Definir una función: 2

```
mientras :: (a -> Bool) -> (a -> a) -> a -> a
```
3

que dada una condición, una función de transformación de estados y un estado inicial devuelva el resultado final que se alcanza aplicando repetidamente la función de transformación mientras se cumpla la condición.

b) Usando la función `mientras`, definir la función que computa el n-ésimo elemento de la sucesión de Fibonacci de manera iterativa. 4

Recordemos que: 5

$$F_0 = 0 \qquad F_1 = 1 \qquad F_{n+2} = F_n + F_{n+1}$$ 6

Árboles binarios infinitos 1

Un **árbol binario** (AB) se representa como un valor del tipo: 2

```
data AB a = Nil | Bin (AB a) a (AB a)
```
3

Un **AB infinito** (ABI) se representa como un valor del tipo: 4

```
data ABI a = IBin (ABI a) a (ABI a)
```
5

Definir una función: 6

```
podadoDesdeElNivel :: Int -> ABI a -> AB a
```
7

de tal modo que `podadoDesdeElNivel` *n a* denota el AB que resulta de podar el árbol *a* a partir del nivel *n*.

Árboles binarios infinitos 1

Consideremos los tipos de datos de las **direcciones**, y los **caminos** 2
(listas finitas de direcciones):

```
data Dirección = Izq | Der 3
type Camino = [Dirección]
```

Un árbol binario infinito cuyos nodos tienen datos de tipo a se 4
puede representar también como una **función de caminos** que,
dado un camino, indica el valor del nodo en dicha posición:

```
type FunciónDeCaminos a = Camino -> a 5
```

Ejercicio 6

a) Definir una función: 7

```
funciónDeCaminosDe :: ABI a -> FunciónDeCaminos a 8
```

que dado un ABI devuelve su función de caminos.

b) Definir la función inversa: 9

```
abiDe :: FunciónDeCaminos a -> ABI a 10
```

que dada una función de caminos devuelve su ABI.

¿ ¿ ¿ ¿ ¿ ¿ ¿ ¿ ¿ ? ? ? ? ? ? ? ? ? 1

Lectura recomendada 2

Capítulo 4 del libro de Bird.

Richard Bird. *Thinking functionally with Haskell*. Cambridge University Press, 2015.

Comentarios: tipos 1

Ojo. Dijimos: 2

"Cada variable se debe usar siempre con un mismo tipo." 3

¿Está bien tipado el siguiente programa? 4

```
sucesor :: Int -> Int
sucesor x = x + 1
```
5

```
opuesto :: Bool -> Bool
opuesto x = not x
```
6

Sí. Hay dos "x" con distinto tipo pero son variables distintas. 7
El programa se podría reescribir así:

```
sucesor x = x + 1
opuesto y = not y
```
8